

# Usual USTBL

## Security Review


Cantina Managed review by:

**Phaze**, Lead Security Researcher
**Windhustler**, Security Researcher



December 17, 2025

# Contents


**1 Introduction** **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

**2 Security Review Summary** **3**

**3 USTBL Token Contract Analysis** **3**

**4 USTBL Actors and Contracts Overview** **5**

**5 `PermissionManager` Permissions Overview** **7**

**6 Redemption Permissions Overview** **10**

**7 USTBL Safe Transaction - USD0 RWA Inclusion** **10**

# 1 Introduction

## 1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

## 1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

## 1.3 Risk assessment

| Severity level | Impact: High | Impact: Medium | Impact: Low |
|---|---|---|---|
| **Likelihood: high** | Critical | High | Medium |
| **Likelihood: medium** | High | Medium | Low |
| **Likelihood: low** | Medium | Low | Low |

### 1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

# 2 Security Review Summary

Usual is a Stablecoin DeFi protocol that redistributes control and redefines value sharing. It empowers users by aligning their interests with the platform's success.

From Sep 12th to Sep 16th the Cantina team conducted a review of core-protocol on commit hash 77302709.

# 3 USTBL Token Contract Analysis

**Summary:** The USTBL Token is a heavily restricted, upgradeable ERC20 token designed for regulated environments. It implements strict transfer whitelisting, making it unsuitable for standard DeFi protocols but appropriate for compliance-heavy use cases like stablecoins.

**On-Chain Deployment Information:**

**Mainnet Deployment:**

- Proxy Address: 0xe4880249745eac5f1ed9d8f7df844792d560e750.
- Implementation Address: 0x15ea0ec460a0e6847ec0aa8d50a84b3a51b95f74.
- Deployment TX: 0x728d46f23e31b485d1a803dc6cc7b98196f2e59fab0d81d7e21a729435acb69b.
- Deployed: April 19, 2024 (Block 19690265).
- Deployer: 0xed608cf38968c0ba09676843d90b33dd59dd7375 (Spiko: Deployer).
- Authority Contract: 0x344278aff344d6fb960705dc18a4912221608472.
- Trusted Forwarder: 0x42DB5A5AF4D82E8eD42405dfA37278c360d923C3.
- Decimals: 5 (highly unusual for stablecoins).

**On-Chain Usage Analysis:**

- Total Transactions: 40 (as of Sep 2025).
- Active Days: 35 days over 1 year 149 days.
- Holder Count: 15 addresses only.
- Usage Pattern: Very infrequent, suggesting limited production use or testing phase.

**Token Distribution:** Top 5 Holders (87.87% concentration):

1. 0x748D361e...99949669D: 54.39% (22.19M tokens).
2. 0xCABF4396...7973fc893: 32.65% (13.32M tokens).
3. 0x0ae8a756...3588e7Cc7: 4.69% (1.91M tokens).
4. 0xdb084AE0...6a08707F6: 2.98% (1.22M tokens).
5. Safe Smart Account: 2.03% (828K tokens).

Notable: High concentration with top 2 holders controlling 87% of supply.

**Findings:**

**1. Transfer Restrictions:** Location: `Token.sol:83-95`.

- Both sender AND receiver must be whitelisted for any transfer to succeed.
- Whitelist validation via `authority.canCall(from, address(this), IERC20.transfer.selector)`.
- Exceptions:
  - Minting (from = address(0)) bypasses sender check.
  - Burning (to = address(0)) bypasses receiver check.
  - Admin can burn from non-whitelisted accounts.
- Impact: Breaks ERC20 composability - incompatible with DEXs, lending protocols, etc...

**2. Permission System:** Components:

- PermissionManaged: Controls ALL admin functions via immutable `authority` contract.
- OwnableUpgradeable: Purely cosmetic - owner has ZERO permissions.
- Restricted functions: `mint()`, `burn()`, `pause()`, `unpause()`, `setOwnership()`, upgrade authorization.

**Notable Features:**

**ERC1363 "Payable Token" Extension:**

- Implements `transferAndCall()`, `transferFromAndCall()`, `approveAndCall()`.
- Enables callback execution on token operations.
- Concern: Uses `require()` instead of custom errors (gas inefficient).
- Risk: Potential reentrancy through callbacks.

**Meta-Transaction Support (ERC2771):**

- Gasless transactions via trusted forwarder.
- Limitation: Meta-transactions DO NOT work with `restricted()` admin functions.
- Comment at `Token.sol:19-20` explicitly states this limitation.
- MulticallUpgradeable is protected against ERC2771 data spoofing.

**UUPS Upgradeability:**

- Only authority can authorize upgrades.
- Uses `_checkRestricted(UUPSUpgradeable.upgradeToAndCall.selector)`.
- Standard implementation with proper storage considerations.

**Additional Features:**

- Pausable: Can freeze all transfers globally.
- ERC20Permit: Gasless approvals via EIP-712 signatures.
- Multicall: Batch transaction support.
- Custom Decimals: Configurable during initialization (stored at slot `m_decimals`).

**Token Security Concerns:**

1. Centralization: Authority has unlimited power (mint/burn/pause/upgrade).
2. No Supply Cap: Unlimited minting capability.
3. ERC1363 Reentrancy: Callback pattern may enable reentrancy attacks.

**Non-Standard ERC20 Behaviors:**

1. Transfer Restrictions: Not freely transferable (breaks ERC20 assumption).
2. ERC1363 Callbacks: Transfer and call pattern.

**Unusual Design Choices:**

1. 5 Decimals: Highly unusual for USD stablecoins (typically 6 or 18).
2. Minimal Usage: Only 40 transactions in 1+ year suggests testing/limited pilot.
3. Liquidity Concentration: 87% held by 2 addresses indicates centralized custody.
4. Powerless Owner: Owner role exists but has no permissions - likely for compliance display.

**Conclusion:** USTBL is a highly controlled token implementation prioritizing regulatory compliance over decentralization. The transfer restrictions make it incompatible with standard DeFi protocols but suitable for regulated financial products. The centralized control presents significant trust assumptions that must be taken into account.

# 4 USTBL Actors and Contracts Overview

**Key Actors and Contracts:** https://dune.com/spiko/dashboard.

**1. USTBL Token Contract (Main Token):** Proxy Address: 0xe4880249745eac5f1ed9d8f7df844792d560e750. Implementation Address: 0x15ea0ec460a0e6847ec0aa8d50a84b3a51b95f74.

- Role: The primary USTBL stablecoin token.
- Type: ERC1967 UUPS upgradeable proxy with ERC20/ERC1363/ERC2771 functionality.
- Features:
  - Strict transfer whitelisting (both sender and receiver must be whitelisted).
  - Meta-transaction support via trusted forwarder.
  - Pausable transfers.
  - UUPS upgradeability.
  - 5 decimals (unusual for stablecoins).
- Total Supply: ~40.8M tokens across 15 holders.
- Deployed: April 19, 2024.
- Risk Level: LOW - Relatively standard token implementation with minor modifications.

**2. Authority/Permission Manager Contract:** Address: 0x344278aff344d6fb960705dc18a4912221608472.

- Role: Complete administrative control over USTBL token via group-based permissions.
- Type: Immutable PermissionManager with group-based access control.
- Powers:
  - Authorize all admin functions (mint/burn/pause/upgrade).
  - Manage group memberships and permissions.
  - Control function-level access via group requirements.
- Group Structure:
  - Group 0 (Admin): Supreme control (always included in requirements).
  - Group 1: Pause/unpause permissions.
  - Group 2: Mint permissions.
  - Group 4: Burn permissions.
  - Group 5: Admin rights over Group 6.
  - Group 6: Transfer permissions.
- Immutable: Cannot be changed after deployment.
- Risk Level: CRITICAL - Single point of failure for all token operations.

**2.1. Redemption Contract (Group 4 - Burn Authority):** Address: 0xda5599f04e9b437c8394b0c2bc68b502a66ebfe8.

- Role: USTBL burning and redemption processing.
- Type: Proxy contract (Redemption contract).
- Powers:
  - Can burn USTBL tokens (Group 4).
  - Can transfer USTBL tokens (Group 6).
- Function: Handles user redemptions by burning USTBL tokens.
- Risk Level: MEDIUM - Contract-based, but critical for redemption flow.

**3. Trusted Forwarder:** Address: 0x42DB5A5AF4D82E8eD42405dfA37278c360d923C3.

- Role: Enables meta-transactions (gasless transactions).
- Function: Pure signature validation and forwarding - no privileged access.
- Limitations: Cannot forward calls to `restricted()` admin functions.
- Usage: Allows users to interact with token without holding ETH for gas.
- Risk Level: LOW - Contract itself poses minimal risk (only validates signatures).

**4. Deployer (Spiko):** Address: `0xed608cf38968c0ba09676843d90b33dd59dd7375`.

- Role: Initial contract deployer.
- Actions: Deployed proxy and initialized token on Apr 19, 2024.
- Current Power: None after deployment.
- Risk Level: LOW - Historical role only.

**5. Top Token Holders:** Note: Top 4 holders are likely Caceis custodians. These appear to be Externally Owned Accounts (EOAs) rather than multisigs, which presents slight additional risk for USTBL custody security.

**Holder #1 (54.39% - 22.19M tokens):** Address: `0x748d361eac241f0b4c2ea789609dea199949669d`.

- Concentration: Over half of total supply.
- Type: Likely Caceis custodian EOA.

**Holder #2 (32.65% - 13.32M tokens):** Address: `0xcabf4396050340fcc6383a54f4680b77973fc893`.

- Combined with #1: 87% of total supply.
- Type: Likely Caceis custodian EOA.

**Holder #3 (4.69% - 1.91M tokens):** Address: `0x0ae8a7565ed1981abd4bb5eaeeca9d43588e7cc7`.

- Type: Likely Caceis custodian EOA.

**Holder #4 (2.98% - 1.22M tokens):** Address: `0xdb084ae0fa9ff14672bdf91cbdb16ff6a08707f6`.

- Type: Likely Caceis custodian EOA.

**Holder #5 (2.03% - 828K tokens):** Address: `0x4fe1329138a17a07d44858a328a2072b42a68092`.

**Holder #6 (1.17% - 477K tokens):** Address: `0xa46ae19a402f44dae6562e472610fb4426298616`.

**Holder #7 (0.72% - 294K tokens):** Address: `0xf831db488e7d56143038067b0b1c79db39b96e00`.

**Holder #8 (0.70% - 286K tokens):** Address: `0x32bf7230786415666d596d664b215b636b97fb36`.

**Holder #9 (0.27% - 110K tokens):** Address: `0x75cb1bd539ee2cb22031603a7b437f1f7077b05e`.

**Holder #10 (0.24% - 99K tokens):** Address: `0x04a8f552e6d13fd00def492d243198e841a8f107`.

**Holder #11 (0.13% - 53K tokens):** Address: `0xb30612580b9e02f72448de8d44e83d806751d381`.

**Holder #12 (0.03% - 10K tokens):** Address: `0x1e2b3e14148487d26923beecb94b251c0a09ba5d` (sherlock.eth).

**Holder #13 (0.002% - 967 tokens):** Address: `0x4c399948a80c9ad1c9712f59d1ccc5f3742ac887`.

**Holder #14 (0.0001% - 49 tokens):** Address: `0x5a450f7599e74de26fc5e4aa2cbaaf423649b9e4`.

**Holder #15 (0.00003% - 10 tokens):** Address: `0xc050b14efea76d16f4a06a16b3772e087237bd0c`.

**Observations:**

**Power Concentration:**

1. Authority contract has absolute control over token functionality.
2. Top 2 holders control 87% of circulating supply.
3. 15 total holders indicates extremely limited distribution.

**Systemic Risks:**

1. **CRITICAL:** Admin key in cloud - Primary operator key appears stored in cloud infrastructure.

2. Authority compromise = complete token control loss.
3. Holder concentration enables market manipulation.
4. Immutable authority prevents governance evolution.
5. Limited adoption (40 transactions in 1+ year).
6. API-based key management creates additional attack vectors.

**Regulatory Implications:**

- Extreme centralization suggests compliance-first design.
- Limited holder count indicates restricted access (likely KYC/whitelist).
- Large holder concentration typical of institutional custody arrangements.

**Oracle Integration:**

**USTBL NAV Chainlink Feed:** Address: `0x477e363c51Ab0C4D13B22CD6B57D56d4a3Cb7Abe`.

- Type: NAVLink (Custom Chainlink Feed).
- Operators: 10 operators.
- Decimals: 6.
- Deviation Threshold: $1^{-7}$ (extremely tight) - triggers oracle updates even when no visible on-chain change occurs due to price feed's 6 decimals.
- Heartbeat: Present.
- Purpose: Provides Net Asset Value pricing for USTBL.
- Data Source Centralization Risk: All 10 operators respond flawlessly at nearly identical times with the same answer, suggesting they likely use the same single API.
- Infrastructure Risk: Operators may run on same server/provider (unverified).
- Expected Behavior: Caceis serves as both custodian and feed data provider, explaining synchronized responses.
- Risk: Price feed dependency and potential single point of failure in data sourcing.

# 5 `PermissionManager` Permissions Overview

**`PermissionManager` Contract Overview:** The USTBL token contract at `0x344278aff344d6fb960705dc18a4912221608472` uses an immutable `PermissionManager` authority that controls all access via group-based permissions.

Key Features:

- Group-based access control with admin hierarchies.
- Function-level permissions mapped to specific groups.
- Controls mint/burn, pause/unpause, transfers, and upgrades.
- Upgradeable proxy with mask-based group system.

Group Structure:

- Group 0 (Admin): Can do everything (always included in requirements).
- Group 1: Can pause/unpause USTBL.
- Group 2: Can mint USTBL.
- Group 4: Can burn USTBL.
- Group 5: Has admin rights over Group 6.
- Group 6: Can transfer USTBL.

Risk Level: CRITICAL - Centralized control point for all token operations.

**Key Addresses:**

**Admin Multisig (Group 0):** Address: `0x4eaa52b6280593a8820c45ce38f974290ae52cea`.

- Role: Supreme admin with control over all functions and groups.
- Type: 2-of-6 multisig contract.
- Powers:
  - Can call ANY function on USTBL (always included in requirements).
  - Default admin rights over all other groups.
  - Can modify group memberships and requirements.
- Risk Level: CRITICAL - Compromise = complete system control.

Signers:

- `0x6c00Be5d1013a5bca938EF6aa8F1E5e706FF384d`.
- `0x4D1d9C5B5d9Ca448194313D799De374f362Bd460`.
- `0xDa47Bd72438F11bef7Cc54F18cd02426dd86A5a2`.
- `0xe67D2cbeaE3090bE017C66e7A6943961619a5D35`.
- `0x4896F23FF83242D8D4EB5fA1dEdF8c57e9145949`.
- `0xA6e06dfE42064B7Cd6DC902a62e74ca5Bb574011`.

Threshold: 2/6 (low).

Signer Activity Analysis:

- Active signers: Transactions are primarily created and executed only by `0x4896F23FF83242D8D4EB5fA1dEdF8c57e9145949` and `0xe67D2cbeaE3090bE017C66e7A6943961619a5D35`.
- Inactive signers:
  - `0x6c00Be5d1013a5bca938EF6aa8F1E5e706FF384d`.
  - `0x4D1d9C5B5d9Ca448194313D799De374f362Bd460`.
  - `0xDa47Bd72438F11bef7Cc54F18cd02426dd86A5a2`.

  These addresses have 0 transaction history/activity and no funds.
- Funding patterns: `0x4896F23FF83242D8D4EB5fA1dEdF8c57e9145949` was funded by `0xe67D2cbeaE3090bE017C66e7A6943961619a5D35` at time of safe creation.
- Limited activity: `0xA6e06dfE42064B7Cd6DC902a62e74ca5Bb574011` was funded by `0xe67d2cbeae3090be017c66e7a6943961619a5d35` at time of safe creation and has only ever called execTransaction on safe.

Decentralization Risk: CRITICAL - Minimal decentralization as all signers with funds are funded from the same source, low threshold, and little data to support that these are active, independent addresses.

**Operations Multisig (Group 1 - Pause Control):** Address: `0xa4f97076721c78b050c2cd6a1bf7b442d801fb4a`.

- Role: Operational control over USTBL pause/unpause functions.
- Type: 1-of-6 multisig contract(**LOW THRESHOLD**).
- Powers:
  - Can pause USTBL transfers (`pause()`).
  - Can unpause USTBL transfers (`unpause()`).
- Risk Level: HIGH - Single signer can halt USTBL operations.

Signers: **IDENTICAL TO ADMIN MULTISIG** - Same 6 addresses control both contracts:

- `0x4D1d9C5B5d9Ca448194313D799De374f362Bd460`.

- 0x6c00Be5d1013a5bca938EF6aa8F1E5e706FF384d.
- 0xDa47Bd72438F11bef7Cc54F18cd02426dd86A5a2.
- 0xe67D2cbeaE3090bE017C66e7A6943961619a5D35.
- 0x4896F23FF83242D8D4EB5fA1dEdF8c57e9145949.
- 0xA6e06dfE42064B7Cd6DC902a62e74ca5Bb574011.

Threshold: 1/6 (extremely low - any single signer can execute).

Critical Risk: The same individuals who control admin functions also control operational pause/unpause, eliminating separation of duties.

**Minting EOA (Group 2, 3 and 5 - Mint Authority):** Address: 0x2B3755c065df5C2805BE8E69C1496DC7d67f6155.

- Role: Primary USTBL minting authority.
- Type: Externally Owned Account (EOA).
- Powers:
  - Can mint USTBL tokens (`mint()`).
  - Member of Groups 2, 3 and 5.
- Risk Assessment:
  - HIGH - Cloud storage = potential key exposure.
  - Single point of failure for all admin operations.
  - API compromise could lead to unauthorized minting.
  - No offline/hardware security apparent.
- Dual Function:
  - Direct admin operations (mint, burn, etc.).
  - ERC2771 meta-transaction forwarding.
- **SECURITY ISSUE:** This key appears to be stored in cloud infrastructure behind API endpoints.
- Risk Level: CRITICAL - EOA with mint powers, potential private key exposure.

**Redemption Contract (Group 4 - Burn Authority):** Address: 0xda5599f04e9b437c8394b0c2bc68b502a66ebfe8.

- Role: USTBL burning and USTBL transfer authority.
- Type: Proxy contract (Redemption contract).
- Powers:
  - Can burn USTBL tokens (`burn()`).
  - Can transfer USTBL tokens (`transfer()`) - Group 6 membership.
- Risk Level: MEDIUM - Contract-based, but critical for redemption flow.

```
Final Permission Groups:
- Group 0: 1 member
(0x4eaa52b6280593a8820c45ce38f974290ae52cea)
- Group 1: 1 member
(0xa4f97076721c78b050c2cd6a1bf7b442d801fb4a)
- Group 2: 1 member
(0x2b3755c065df5c2805be8e69c1496dc7d67f6155)
- Group 3: 1 member
(0x2b3755c065df5c2805be8e69c1496dc7d67f6155)
- Group 4: 1 member
(0xda5599f04e9b437c8394b0c2bc68b502a66ebfe8)
- Group 5: 1 member
(0x2b3755c065df5c2805be8e69c1496dc7d67f6155)
- Group 6: 57 members (various addresses)
```

# 6 Redemption Permissions Overview

**Redemption Contract Overview:** The Redemption contract at `0xDA5599f04e9b437C8394b0c2BC68B502A66ebFe8` manages the burning process for USTBL tokens. Users can initiate redemptions by transferring tokens to the contract, which creates a pending redemption with a 14-day execution window. Authorized addresses can then execute the redemption (burning the input tokens), or users can cancel expired redemptions to get refunds.

Group Structure:

- Group 0 (Admin): Can call `executeRedemption()`, `registerOutput()`, `setMinimum()`, `upgradeToAndCall()`.
- Group 2: Can call `executeRedemption()`.

**Key Addresses with Access:**

**Admin Multisig (Groups 0 and 255):** Address: `0x4EAA52B6280593a8820C45cE38f974290Ae52Cea`.

- Powers: Can execute redemptions, configure outputs/minimums, and upgrade contract.
- Type: 2-of-6 multisig.

**EOA (Groups 2, 3, 5, and 255):** Address: `0x2B3755c065df5C2805BE8E69C1496DC7d67f6155`.

- Powers: Can execute redemptions only.
- Type: Externally Owned Account.
- Risk: EOA with redemption execution powers.

# 7 USTBL Safe Transaction - USD0 RWA Inclusion

**Overview:** Adds Spiko US T-Bills Money Market Fund (`USTBL`) as an eligible USD0 RWA collateral and initializes its NAV oracle in the Usual protocol.

- Safe transaction: Gnosis Safe batch.
- USTBL token (proxy): `0xe4880249745eAc5F1eD9d8F7DF844792D560e750`.
- USTBL NAV Chainlink feed: `0x477e363c51Ab0C4D13B22CD6B57D56d4a3Cb7Abe`.

After execution, `DaoCollateral` accepts USTBL in `swap`, `redeem`, and `redeemDao`, gated by `TokenMapping.isUsd0Collateral(rwa)` and priced via `ClassicalOracle`.

**Safe Batch Tx Contents (2 calls):**

**1. Register USTBL as USD0 RWA collateral:**

- Target contract: `TokenMapping` (proxy).
- Function: `addUsd0Rwa(address rwa)`.
- Params: `rwa` = `0xe4880249745eAc5F1eD9d8F7DF844792D560e750`.
- Effect: Marks USTBL as eligible collateral (`isUsd0Collateral[rwa] = true`).

**2. Initialize USTBL NAV oracle:**

- Target contract: `ClassicalOracle` (proxy).
- Function: `initializeTokenOracle(address token, address dataSource, uint64 timeout, bool isStablecoin)`.
- Params:
  - `token` = `0xe4880249745eAc5F1eD9d8F7DF844792D560e750`.
  - `dataSource` = `0x477e363c51Ab0C4D13B22CD6B57D56d4a3Cb7Abe`.
  - `timeout` = `345600` (4 days).
  - `isStablecoin` = `false`.

- Effect: Registers the Chainlink NAV feed for USTBL with a 4-day staleness timeout and non-stablecoin pricing semantics.

**Affected Components:**

- `TokenMapping.sol`: eligibility gate for RWA collateral.
- `ClassicalOracle.sol` / `AbstractOracle.sol`: price source and normalization.
- `DaoCollateral.sol`: uses mapping + price to compute mint/redeem amounts and apply fees/CBR.

**Decimals and Oracle Handling:**

- USTBL token `decimals`: 5.
- USTBL NAV feed `decimals`: 6.

The system normalizes price and amount independently, then multiplies them by 18 decimals:

1. Oracle returns price in feed decimals; it is scaled to 18 by `AbstractOracle.getPrice`:

```
// AbstractOracle.sol
function getPrice(address token) public view override returns (uint256) {
    (uint256 price, uint256 decimalsPrice) = _latestRoundData(token);
    price = price.tokenAmountToWad(uint8(decimalsPrice)); // e.g., 6 -> 18 (x 1e12)
    _checkDepegPrice(token, price);
    return price; // 18 decimals
}
```

2. `DaoCollateral` converts the token amount from its own `decimals()` to 18 using `Normalize.tokenAmountToWad` before applying the price:

```
// DaoCollateral.sol
function _getQuoteInUsd(uint256 tokenAmount, address rwaToken)
    internal
    view
    returns (uint256 wadAmountInUSD)
{
    (uint256 wadPriceInUSD, uint8 decimals) = _getPriceAndDecimals(rwaToken);
    uint256 wadAmount = tokenAmount.tokenAmountToWad(decimals); // e.g., 5 -> 18 (x 1e13)
    wadAmountInUSD = Math.mulDiv(wadAmount, wadPriceInUSD, SCALAR_ONE, Math.Rounding.Floor);
}
```

3. For USD → token conversions (redemptions), it inverts using `Normalize.wadTokenAmountForPrice`, returning token amounts in the token's native `decimals` (e.g., 5 for USTBL):

```
// Normalize.sol
function wadTokenAmountForPrice(uint256 wadStableAmount, uint256 wadPrice, uint8 tokenDecimals)
    internal
    pure
    returns (uint256)
{
    return Math.mulDiv(wadStableAmount, 10 ** tokenDecimals, wadPrice, Math.Rounding.Floor);
}
```

Therefore:

- Feed 6 decimals is scaled to 18 in the price path.
- Token 5 decimals is scaled to 18 in the amount path.
- Multiplication and division happen entirely in 18 decimals.